Bitácora de uso de IA · Parcial 01

Equipo 02 · Mathias Carstens, Mikel Muñoz, Leonardo Reyes, Federico Celis, Fernando Álvarez · Tópicos Especiales para la Gestión de Datos · Prof. Armen Djenanian

> La IA se usó como **asistente de desarrollo** (Claude) y como **componente del producto** (LLM vía OpenRouter dentro del agente). Ningún resultado de la IA se aceptó sin una validación técnica verificable: ejecución en PostgreSQL, conciliación independiente con pandas, pruebas unitarias, matriz de pruebas y revisión visual. Las entradas registran también los resultados que se modificaron o rechazaron. Pendiente del equipo: ejecutar el agente con la API key en su .env (python scripts/ejecutar_pruebas.py --modo auto), que llama al LLM pregunta por pregunta, y registrar el resultado.

Resumen de decisiones

#	Componente	Decisión
1	Planificación	Aceptar
2	Dataset (fuente operacional)	Modificar
3	Dataset / SCD2	Modificar
4	Data Mart (sql/Datamart.sql)	Aceptar
5	Decisión SCD	Aceptar
6	BDOO (agente/bdoo/)	Modificar
7	Intérprete por reglas	Modificar
8	Intérprete LLM (OpenRouter) y regla RV11	Aceptar con reserva
9	Seguridad	Aceptar
10	Pruebas (Pruebas.xlsx)	Modificar
11	Diagramas	Rechazar (arquitectura)
12	Informe técnico y bitácora	Aceptar con revisión del equipo
13	Agente en ejecución (uso de IA dentro del producto)	Aceptar
14	Revisión independiente del agente	Modificar
15	Intérprete LLM: validación con OpenRouter real y anclaje	Modificar
16	Matriz de pruebas en vivo con la API key del equipo	Modificar

Entradas

Entrada 1 · Planificación	29-10-2026
Prompt utilizado	«Vamos a diseñar y ejecutar un plan para realizar esta asignatura. Dame todas las dudas que tengas al respecto» (adjuntos: enunciado del Parcial 01 y anuncio de Canvas con fechas).
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Lista de 14 dudas priorizadas (dataset, motor de la BDOO, motor del Data Mart, LLM, equipo, reparto, interfaz, SCD, preguntas adicionales, fecha de la presentación) con una propuesta por defecto para cada una.
Componente afectado	Planificación
Validación técnica	El equipo contrastó las dudas con el enunciado (secciones 4–11) y con las presentaciones P04 (DW) y P06 (BDOO) del docente.
Decisión	**Aceptar**
Justificación	Respondimos: el docente no dio dataset (crear uno propio), usar OpenRouter como LLM, mismo equipo de proyecto.

Entrada 2 · Dataset (fuente operacional)	29-10-2026
Prompt utilizado	«No dio un dataset, supongo que podemos crear nosotros nuestro propio dataset con la estructura conveniente y apropiada. Te mando presentaciones del profesor. Para el LLM podemos usar OpenRouter… haz todo el plan tú para ejecutarlo desde acá»
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Generador determinista data/generar_dataset.py (semilla fija) de una empresa ficticia con errores de calidad sembrados.
Componente afectado	Dataset (fuente operacional)
Validación técnica	Se revisaron con pandas las distribuciones por mes, categoría, canal y sucursal. Hallazgos: Tecnología dominaba el ingreso porque el canal mayorista compraba entre 6 y 48 laptops por línea, y los precios por rango de categoría producían casos absurdos (un mouse más caro que una laptop).
Decisión	**Modificar**
Justificación	Se corrigieron las cantidades mayoristas por tipo de producto y se fijó un precio de referencia por producto (±12 % por marca), para que las conclusiones de negocio fueran verosímiles.

Entrada 3 · Dataset / SCD2	29-10-2026
Prompt utilizado	(misma sesión) Ejecución del script SQL sobre el dataset generado.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) + PostgreSQL 16
Resultado obtenido	La carga falló: la restricción CHECK ck_cliente_vigencia rechazó una versión de cliente con fecha_fin < fecha_inicio.
Componente afectado	Dataset / SCD2
Validación técnica	Se rastreó al generador: algunas reclasificaciones de segmento caían antes de la fecha de alta del cliente.
Decisión	**Modificar**
Justificación	Se exigió que la reclasificación ocurra al menos 30 días después del alta. La restricción del modelo funcionó como control de calidad y se mantiene.

Entrada 4 · Data Mart (sql/Datamart.sql)	29-10-2026
Prompt utilizado	(misma sesión) Construir el Data Mart: esquema estrella, grano, claves sustitutas, desconocidos, SCD, ETL, rol de solo lectura, controles y consultas parametrizadas.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Script SQL único e idempotente: esquemas oltp/dm/control, 5 dimensiones + hecho, miembros −1/−2, SCD2, ETL, rechazos, rol agente_lector, 3 vistas de control y 10 consultas PREPARE.
Componente afectado	Data Mart (sql/Datamart.sql)
Validación técnica	Ejecución completa en PostgreSQL 16: conciliación 7/7 OK, integridad 10/10 con 0 hallazgos y conciliación independiente con pandas sobre los CSV 5/5 OK (ingreso neto idéntico al centavo).
Decisión	**Aceptar**
Justificación	Los controles demuestran que no hay pérdida ni duplicación de ingresos, cantidades ni tickets.

Entrada 5 · Decisión SCD	29-10-2026
Prompt utilizado	(misma sesión) Justificar SCD Tipo 1 vs Tipo 2 para el segmento del cliente y su efecto en el agente.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Recomendación de SCD Tipo 2 y una consulta Q4-bis que simula el Tipo 1 usando el segmento actual.
Componente afectado	Decisión SCD
Validación técnica	Se compararon ambas respuestas sobre el Data Mart: con Tipo 2 el segmento que más cae es Corporativo (−20,6 %); con Tipo 1 sería Frecuente (−31,5 %) y Corporativo parecería crecer (+5,5 %).
Decisión	**Aceptar**
Justificación	La evidencia numérica muestra que el Tipo 1 llevaría al agente a una conclusión falsa.

Entrada 6 · BDOO (agente/bdoo/)	29-10-2026
Prompt utilizado	(misma sesión) Implementar las 10 clases mínimas como objetos persistentes, con herencia, encapsulamiento, polimorfismo, composición y persistencia, siguiendo los conceptos de la clase P06.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) + ZODB
Resultado obtenido	Modelo ZODB con 10 clases base, 18 subclases, TDA con validación (equivalente al TDA DATOS/RIF de Oracle), Blob como LOB y versionamiento.
Componente afectado	BDOO (agente/bdoo/)
Validación técnica	Pruebas unitarias (clases abstractas, máquina de estados, TDA) y evidencia generada tras cerrar y reabrir la base. Hallazgo: los IDO salían vacíos porque ZODB los asigna al confirmar la transacción.
Decisión	**Modificar**
Justificación	Se agregó un savepoint antes de mostrar los objetos, para que los IDO estén disponibles en la respuesta.

Entrada 7 · Intérprete por reglas	29-10-2026
Prompt utilizado	(misma sesión) Intérprete en español que funcione sin internet como respaldo del LLM.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Parser por reglas (métricas, periodos, dimensiones, valores, ambigüedad, parámetros faltantes, seguimiento).
Componente afectado	Intérprete por reglas
Validación técnica	Se probó con más de 50 preguntas. Errores detectados: (a) «¿cuáles son los canales… del segmento Premium?» devolvía límite 1 por la palabra «el segmento»; (b) «¿cómo evolucionó…?» se tomaba como comparación de periodos; (c) «canal Instagram» se ignoraba en silencio y devolvía el total; (d) «vendimos» no se reconocía como venta.
Decisión	**Modificar**
Justificación	Se corrigieron los cuatro casos. El (c) era el más grave porque la respuesta parecía válida: ahora el valor llega a la regla RV05, que pide aclaración y lista los valores válidos.

Entrada 8 · Intérprete LLM (OpenRouter) y regla RV11	29-10-2026
Prompt utilizado	(misma sesión) Integrar OpenRouter sin que el LLM pueda inventar cifras ni generar SQL.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	El LLM sólo devuelve un JSON con códigos del catálogo; la narrativa se acepta sólo si todas sus cifras están en el resultado (RV11).
Componente afectado	Intérprete LLM (OpenRouter) y regla RV11
Validación técnica	El entorno de desarrollo no tenía acceso a openrouter.ai, así que se probó con respuestas simuladas: JSON válido, JSON inválido (cae a reglas), valor de filtro inexistente, dimensión inexistente, narrativa con cifras inventadas (rechazada) y narrativa respaldada (aceptada).
Decisión	**Aceptar con reserva**
Justificación	Faltaba la prueba con el LLM real: se hizo en la entrada 15.

Entrada 9 · Seguridad	29-10-2026
Prompt utilizado	(misma sesión) Impedir acceso a tablas no autorizadas, instrucciones destructivas y campos inexistentes.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Defensa en cuatro capas: RV01 (texto), listas blancas + parámetros, RV07 (SQL) y rol de solo lectura.
Componente afectado	Seguridad
Validación técnica	Pytest con intentos de inyección (DROP, DELETE, comentarios, pg_sleep, oltp.*, control.*, valor malicioso como filtro): todos bloqueados o tratados como parámetro.
Decisión	**Aceptar**
Justificación	Cumple la sección 6.2 del enunciado aunque se use SQL dinámico.

Entrada 10 · Pruebas (Pruebas.xlsx)	29-10-2026
Prompt utilizado	(misma sesión) Matriz de pruebas con los casos obligatorios de la sección 7.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	25 casos ejecutados automáticamente (scripts/ejecutar_pruebas.py), con el valor esperado calculado por separado con pandas.
Componente afectado	Pruebas (Pruebas.xlsx)
Validación técnica	25/25 OK en la primera versión. La revisión independiente (entrada 14) mostró que P10 compartía un error con el agente: ambos agrupaban clientes por nombre.
Decisión	**Modificar**
Justificación	Se corrigió P10 y se amplió la matriz a 35 casos (P25–P34 son regresiones de la revisión).

Entrada 11 · Diagramas	29-10-2026
Prompt utilizado	(misma sesión) Diagramas de estrella, de clases y de arquitectura.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) + Mermaid
Resultado obtenido	Diagramas Mermaid de estrella y de clases (legibles); el de arquitectura salió ilegible por el cruce de flechas.
Componente afectado	Diagramas
Validación técnica	Revisión visual de cada imagen.
Decisión	**Rechazar (arquitectura)**
Justificación	Se rehízo a mano el diagrama de arquitectura como SVG. Los otros dos se aceptaron.

Entrada 12 · Informe técnico y bitácora	29-10-2026
Prompt utilizado	(misma sesión) Redactar el informe técnico a partir de las evidencias reales.
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código)
Resultado obtenido	Generador docs/informe/build_informe.py que toma las cifras de los JSON de evidencia (no se transcriben a mano).
Componente afectado	Informe técnico y bitácora
Validación técnica	Revisión visual página por página del PDF. Se corrigió la portada, que se desbordaba a una segunda página.
Decisión	**Aceptar con revisión del equipo**
Justificación	Cada integrante debe leer la sección de su componente antes de la defensa.

Entrada 13 · Agente en ejecución (uso de IA dentro del producto)	29-10-2026
Prompt utilizado	Prompt de sistema en agente/interpretes/llm.py: catálogo de métricas y dimensiones, valores válidos, rango de fechas y 19 reglas de interpretación; salida JSON obligatoria.
Herramienta empleada	OpenRouter · modelo configurable (por defecto openai/gpt-4o-mini, temperatura 0)
Resultado obtenido	Intención estructurada y una interpretación gerencial breve.
Componente afectado	Agente en ejecución (uso de IA dentro del producto)
Validación técnica	Automática en cada pregunta: anclaje a la lectura literal, construir_intencion() + RV02–RV06 sobre la intención y RV11 sobre la narrativa.
Decisión	**Aceptar**
Justificación	Si el LLM falla, el intérprete por reglas garantiza el funcionamiento.

Entrada 14 · Revisión independiente del agente	29-10-2026
Prompt utilizado	Encargo a un agente revisor de Claude que no participó en la construcción: «Revisa como el profesor más exigente: verifica cada requisito del enunciado, prueba el agente con unas 40 preguntas variadas en español (paráfrasis, periodos, comparaciones, fuera de alcance, operaciones no autorizadas), contrasta cifras con SQL o pandas y lista los defectos».
Herramienta empleada	Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) (agente revisor independiente)
Resultado obtenido	Más de 120 preguntas probadas. Defectos graves: ranking de clientes agrupado por nombre (sumaba empresas distintas), división entera en variaciones de unidades y tickets, periodos mal leídos («T1 2026», «último semestre», «enero a marzo»), año parcial comparado contra año completo, preguntas fuera de alcance respondidas con cifras («tickets anulados», «proyección»), miembros «Sin correspondencia» encabezando rankings y la regla RV01 bloqueando preguntas de negocio («¿qué productos debemos eliminar del catálogo?»).
Componente afectado	Revisión independiente del agente
Validación técnica	Cada defecto se reprodujo y se verificó contra SQL o pandas. Se agregó una prueba de regresión para cada uno (P25–P34 y 6 pruebas unitarias). En la segunda pasada (unas 75 preguntas nuevas, 7 cifras verificadas con pandas) aparecieron otros defectos: clientes nombrados en la pregunta, seguimientos («¿y por canal?», «¿y en los tres meses anteriores?», «¿y el segundo?»), «frente al de 2025» y verbos como «facturamos». También se corrigieron.
Decisión	**Modificar**
Justificación	Los defectos producían respuestas incorrectas con apariencia confiable, que es el peor caso para un agente gerencial. La matriz pasó de 25 a 35 casos, todos correctos.

Entrada 15 · Intérprete LLM: validación con OpenRouter real y anclaje	29-10-2026
Prompt utilizado	Prompt de sistema del agente (agente/interpretes/llm.py) más las preguntas en lote, enviado a openai/gpt-4o-mini en el chat de OpenRouter con la cuenta del equipo: la matriz completa (dos versiones del prompt) y dos conjuntos de preguntas nuevas (14 y 12) que no están en la matriz.
Herramienta empleada	OpenRouter · openai/gpt-4o-mini (respuestas reales) + Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) para el análisis y el código
Resultado obtenido	El LLM solo acertó 22/33 y 17/34 casos de la matriz. Inventó periodos («ingreso neto por mes» → últimos 12 meses), inventó los canales de «compare dos canales», tomó «ventas» como venta bruta o unidades, puso valores en la dimensión equivocada, devolvió la operación en el campo tipo y marcó preguntas inocentes como operación no autorizada. Reforzar el prompt (reglas 11–19) no lo resolvió: en la segunda corrida aparecieron errores nuevos.
Componente afectado	Intérprete LLM: validación con OpenRouter real y anclaje
Validación técnica	Las respuestas reales se grabaron en docs/evidencias/llm_openrouter/ y se reprodujeron con scripts/validar_llm.py contra la misma verificación de la matriz (oráculo pandas) y, para las preguntas nuevas, contra la interpretación esperada escrita de antemano. En el segundo conjunto nuevo, antes de los ajustes: reglas 11/12, LLM solo 7/12, híbrido 8/12. Después: híbrido 33/33, 34/34, 14/14 y 12/12; matriz por reglas 35/35; pytest 41/41.
Decisión	**Modificar**
Justificación	Se agregó el anclaje (anclar()): el LLM aporta la comprensión semántica y la lectura literal fija los hechos escritos en la pregunta (fechas, valores, sentido del orden). Lo inventado se descarta y queda como supuesto visible. Salvedades: las reglas del anclaje se escribieron después de ver las fallas, así que esos 100 % no son una medida ciega, y las preguntas se enviaron en lote, no una por una como hace el agente. También se corrigió un defecto del agente que apareció aquí: un «total» con dimensión mostraba sólo la primera fila.

Entrada 16 · Matriz de pruebas en vivo con la API key del equipo	30-10-2026
Prompt utilizado	python scripts/ejecutar_pruebas.py --modo auto --equipo 02 en la computadora del equipo, con la key parcial1 de OpenRouter: el agente llama a openai/gpt-4o-mini una vez por pregunta, con el contexto de la sesión.
Herramienta empleada	OpenRouter · openai/gpt-4o-mini + Claude (Anthropic) · modelo Claude Opus 5.5 · app Claude (asistente de desarrollo con ejecución de código) para el diagnóstico
Resultado obtenido	32/35. P04 y P29: el LLM pidió un periodo que no hacía falta (en P29 el periodo, 2025, estaba en la pregunta) y el agente pidió aclaración. P23b («¿y en 2026?»): la cifra fue correcta, pero se perdió la marca de pregunta de seguimiento.
Componente afectado	Matriz de pruebas en vivo con la API key del equipo
Validación técnica	Se revisó la intención detectada y las reglas activadas de cada falla (docs/evidencias/pruebas.json). Detalle en docs/evidencias/llm_openrouter/matriz_en_vivo_2026-10-30.md. Tras el ajuste: dos pruebas unitarias nuevas (43/43) y una simulación de la matriz con esas tres respuestas reales da 35/35.
Decisión	**Modificar**
Justificación	Si el LLM pide parámetros que la lectura literal sí encuentra, se usa la lectura literal y queda el supuesto visible. En los seguimientos se conserva la marca. Falta repetir la corrida en vivo con el código corregido.